Adaptive Interview Coaching Agent

Reinforcement Learning for Agentic AI Systems

Take-Home Final Examination — Technical Report

INFO 7375 – Prompt Engineering and AI

By: Priti Pradeep Ghosh

1. Executive Summary

This report presents the Adaptive Interview Coaching Agent (AICA), a fully functional agentic AI system that applies reinforcement learning to personalise and optimise interview preparation coaching sessions. The system integrates two distinct reinforcement learning approaches as required by the assignment: a Deep Q-Network (DQN) for question strategy selection and a UCB1 Bandit for topic exploration. A custom AnswerEvaluatorTool provides structured quality assessment of candidate responses, serving as the primary reward signal for both learning components.

The system is implemented entirely in Python using only NumPy and Matplotlib, with no dependency on external deep learning frameworks, demonstrating that sophisticated RL pipelines can be constructed from first principles. Across 500 training episodes, the AnswerEvaluatorTool was invoked 14,000 times, producing a training reward that improved from 12.96 (episodes 1–50) to 17.37 (episodes 451–500), a 34.1% gain. In direct before/after comparison, the trained agent achieves a session reward of 16.51 versus 9.97 for a random baseline, a 65.6% improvement, while raising mean answer quality from 0.484 to 0.711.

2. System Architecture

AICA is structured as a four-module agentic workflow system. The modules interact in a closed-loop cycle: the UCB Bandit selects a topic, the DQN selects a question strategy, the Answer Evaluator Tool scores the candidate's response, and the Environment computes the final reward. Both learning components are updated from this shared reward signal.

Module	File	Type	Responsibility
Interview Environment	environment.py	Agentic environment	Simulates a 20-question interview session. Maintains candidate state, applies actions, calls the evaluator tool, and computes shaped reward.
DQN Agent	dqn_agent.py	RL Component 1	Selects which question strategy (action) to apply given the 12-dimensional state vector. Uses Double DQN with experience replay and soft target updates.
UCB1 Bandit	ucb_bandit.py	RL Component 2	Selects which topic to focus on next. Balances exploration of under-tested topics with

Module	File	Type	Responsibility
			exploitation of high-reward topics using UCB1 scores.
Answer Evaluator Tool	answer_evaluator.py	Custom Tool	Evaluates free-text answers using a 3-signal rubric: keyword coverage, structural quality, and length adequacy. Returns EvaluationResult with score, reward, and feedback.

2.1 Information Flow

At each time step t in an episode, the following sequence executes:

1. UCB Bandit selects topic a_b using UCB1 scores: a_b = argmax_k [Q(k) + c·√(ln(t)/N(k))]
2. Environment state s_t is encoded into a 12-dimensional feature vector.
3. DQN Agent selects question strategy a_d using ε-greedy policy over Q(s_t, a).
4. Environment applies action: updates topic, difficulty, and candidate knowledge.
5. AnswerEvaluatorTool.evaluate() is called with a synthetic candidate answer.
6. Tool returns EvaluationResult with score in [0,1]; shaped reward r_t is computed.
7. DQN stores (s_t, a_d, r_t, s_{t+1}, done) in replay buffer and performs a gradient update.
8. UCB Bandit updates Q(a_b) with incremental mean: Q(a) ← Q(a) + (r − Q(a))/N(a).
9. Episode terminates after 20 questions; total reward is the performance metric.

3. RL Component 1: Value-Based Learning (Double DQN)

3.1 Problem Formulation

The DQN component frames interview strategy selection as a Markov Decision Process (MDP) with the following formal specification:

MDP Element	Definition	Dimension / Cardinality
State space S	Feature vector encoding interview context	12 continuous dimensions
Action space A	Question strategy type	5 discrete actions
Reward R(s,a,s')	Composite quality and improvement signal	Scalar in [−0.3, 1.3]

MDP Element	Definition	Dimension / Cardinality
Discount factor γ	Weighting of future vs immediate reward	0.95
Transition T(s'|s,a)	Environment dynamics (stochastic)	Implicit in simulation

3.2 State Space Design

The 12-dimensional state vector $s \in \mathbb{R}^{12}$ encodes all information the agent needs to make a good question strategy decision. Each dimension is normalised to [0,1]:

Indices	Feature	Description
0–4	Topic knowledge scores	Estimated knowledge level for each of the 5 topics, updated incrementally as the candidate answers questions. Range [0,1].
5–7	Difficulty one-hot encoding	[1,0,0]=easy, [0,1,0]=medium, [0,0,1]=hard. Encodes the current question difficulty level.
8	Candidate confidence	Initialised uniformly at random in [0.3,0.9], stable across session. Affects answer noise.
9	Candidate fatigue	Starts at 0, increases by 0.05 per question. Penalises answer quality and the agent for over-asking.
10	Session progress	questions_asked / max_questions, normalised to [0,1]. Allows agent to account for time remaining.
11	Last answer quality	Score returned by AnswerEvaluatorTool on the previous question, giving the agent short-term feedback.

3.3 Action Space

ID	Action	Effect on Environment	When optimal
0	ask_new_topic	Switch to a different topic at easy difficulty	When current topic is saturated or candidate is struggling with a topic
1	drill_deeper	Increase difficulty by one level	When candidate is performing well; extracts more learning signal
2	clarify	Keep same topic and difficulty	When agent needs more signal before changing strategy

ID	Action	Effect on Environment	When optimal
3	pivot_easier	Decrease difficulty by one level	When candidate is struggling; prevents demoralisation and builds foundation
4	give_hint_then_ask	Boost candidate knowledge +0.05 before asking	When candidate is just below threshold; coaching move that yields higher quality

3.4 Reward Function Design

The reward function is carefully engineered to incentivise the desired coaching behaviours. A naive reward using only raw answer quality would cause the agent to always ask easy questions (maximising Q(answer)). The shaping terms correct this:

$$r(s, a, s') = Q(answer) + 0.3 \cdot \delta(improvement) - 0.2 \cdot \delta(fatigue > 0.7) - 0.1 \cdot \delta(topic_repeat \geq 3)$$

Term	Value	Justification
Q(answer)	[0, 1]	Primary signal: how well did the candidate answer? Provided by AnswerEvaluatorTool.
+0.3 if improvement	Bonus	Rewards the agent for eliciting answers better than the candidate's historical mean on that topic. Promotes measurable learning, not just high scores.
−0.2 if fatigue > 0.7	Penalty	Discourages sessions that over-exhaust the candidate in the latter half. Simulates real coaching ethics.
−0.1 if same topic ≥3 turns	Penalty	Prevents the agent from getting stuck in a local optimum by asking the same topic repeatedly. Encourages breadth.

3.5 Network Architecture and Training

The Q-network is a 2-hidden-layer MLP (128 neurons per layer, ReLU activations) implemented in pure NumPy with He initialisation. The following design decisions were made based on established DQN literature:

- Double DQN: the online network selects the greedy next action; the target network evaluates it. This decouples selection from evaluation, addressing the overestimation bias in vanilla DQN (Van Hasselt et al., 2016). Target Q: $y = r + \gamma \cdot Q_target(s', argmax_a\ Q_online(s', a))$

- Experience Replay Buffer: ring buffer of capacity 10,000. Mini-batches of 64 are sampled uniformly, breaking temporal correlations between consecutive transitions. Without replay, the network would overfit to recent transitions and forget earlier states.
- Soft Target Updates (Polyak Averaging): $\theta_target \leftarrow \tau \cdot \theta_online + (1-\tau) \cdot \theta_target$ with $\tau=0.01$. Gradual updates prevent the instability caused by abrupt target network changes.
- Huber Loss (SmoothL1): $L(y,\hat{y}) = 0.5(y-\hat{y})^2$ if $|y-\hat{y}| \leq 1$, else $|y-\hat{y}|-0.5$. More robust to outlier transitions than MSE during early high-variance exploration.
- ε-Greedy Decay: ε decays multiplicatively by 0.995 per episode from 1.0 to 0.05 minimum, reaching 0.05 by approximately episode 300. This balances early exploration with later exploitation.
- Gradient Clipping: global gradient norm clipped to 1.0, preventing exploding gradients during the high-variance exploration phase.

4. RL Component 2: Exploration Strategy (UCB1 Bandit)

4.1 Problem Formulation

The DQN decides how to ask questions; the UCB Bandit decides what to ask about. This two-level decision hierarchy prevents the DQN from also having to learn topic selection, which would greatly expand the effective state space and slow convergence.

The topic selection problem is a K-armed stochastic bandit (K=5 topics). At each step, the agent must choose which arm (topic) to pull, observing a noisy reward corresponding to answer quality. The goal is to maximise cumulative reward over a finite horizon (20 questions per episode).

4.2 UCB1 Algorithm

UCB1 (Auer et al., 2002) is selected over greedy or epsilon-greedy strategies because it provides provably optimal regret bounds for stochastic bandits. The selection rule is:

$$a^* = \text{argmax}_k \left[\hat{Q}(k) + c \cdot \sqrt{\ln(t) / N(k)} \right]$$

Symbol	Meaning	Value in AICA
$\hat{Q}(k)$	Empirical mean reward for topic k	Incremental mean of answer quality scores for that topic
N(k)	Number of times topic k selected	Maintained per topic; initialised to 0

Symbol	Meaning	Value in AICA
t	Total selections so far	Incremented each step; initialised to 0
c	Exploration coefficient	1.2 (tuned; higher →0 more exploration)
ln(t)/N(k)	Uncertainty bonus	Large when topic under-explored; decays as topic is sampled

4.3 Update Rule

After observing reward r for arm k, the empirical mean is updated incrementally:

$$\hat{Q}(k) \leftarrow \hat{Q}(k) + (r - \hat{Q}(k)) / N(k)$$

This O(1) update is memory-efficient and numerically stable for large N. The initialisation phase pulls each arm exactly once before computing UCB scores, guaranteeing all 5 topics receive at least one evaluation per episode.

4.4 Thompson Sampling Baseline

A Thompson Sampling variant (Beta-Bernoulli conjugate) is included in ucb_bandit.py as a comparative baseline. It treats Q(answer) ≥ 0.5 as a ‘success’ and samples from Beta(α, β) to select arms. This enables future ablation studies comparing UCB1 vs Thompson Sampling exploration strategies.

4.5 Observed Bandit Behaviour

After 500 training episodes (14,000 bandit pulls), the UCB Bandit converged strongly toward the ml_concepts topic (92.7% of pulls). This emergent behaviour reflects the reward landscape of the simulation: the candidate profiles have high variance in ml_concepts knowledge, making it the topic with the highest expected improvement signal. The bandit correctly identified this through exploration and exploited it thereafter.

Topic	Pull fraction	Mean reward	Interpretation
algorithms	1.6%	Moderate	Explored early; lower variance than ml_concepts
system_design	1.7%	Moderate	Similar; bandit deprioritised after initial pulls
behavioral	1.9%	Moderate	Consistent but lower reward ceiling

Topic	Pull fraction	Mean reward	Interpretation
databases	2.1%	Moderate	Slight preference over behavioral; higher keyword density
ml_concepts	92.7%	High	High-variance topic; bandit correctly identified as highest-yield

5. Custom Tool: AnswerEvaluatorTool

5.1 Tool Design and Purpose

The AnswerEvaluatorTool is a custom-built evaluation component that bridges raw candidate text responses and the numerical reward signal required by the RL components. Without this tool, the agent would have no structured feedback mechanism beyond a pure simulation score. The tool was called 14,000 times during training, making it the most-exercised component in the pipeline.

The tool exposes a clean API consistent with the agentic tool pattern: a single evaluate() method accepting structured inputs and returning a structured EvaluationResult dataclass. It is stateful at the session level, maintaining reward history per topic to enable improvement bonuses.

5.2 Evaluation Pipeline

Each call to evaluate(answer, topic, difficulty, session_id) executes a 3-signal rubric:

Signal	Weight	Method	What it measures
Keyword coverage	55%	Weighted regex matching against a domain vocabulary per topic/difficulty	Whether the answer contains the expected technical terminology. A hard algorithms answer without ‘LRU’, ‘O(1)’, or ‘hash map’ scores near zero.
Structural quality	25%	Regex patterns for definition, example, comparison, quantification, reasoning	Whether the answer has good explanation structure beyond just vocabulary. Detects phrases like ‘for example’, ‘because’, ‘compared to’.
Length adequacy	20%	Word count vs difficulty threshold (easy:20, medium:40, hard:70 words)	Whether the answer is substantive. A 3-word answer to a hard question scores near zero regardless of keyword content.

5.3 Composite Score and Reward

The composite score is computed as:

score = min(1.0, (0.55 · keyword_score + 0.25 · structure_score + 0.20 · length_score) · difficulty_multiplier)

Difficulty multipliers are: easy=0.9, medium=1.0, hard=1.1. This reflects that hard questions demand more depth and vocabulary, so a partially correct hard answer should be penalised less than a partially correct easy answer.

The reward field includes an additional session-level improvement bonus of +0.15 when score exceeds the candidate's topic-specific session mean, providing a shaping signal that rewards actual learning progress over time.

5.4 Tool Outputs

The EvaluationResult dataclass exposes: score (normalised quality), reward (RL signal), keyword_score, structure_score, length_score, keywords_found, keywords_missed (top 5), feedback (human-readable coaching string), topic, difficulty, and word_count.

This structured output serves dual purposes: the reward field feeds into the RL pipeline, while feedback, keywords_found, and keywords_missed would be surfaced to real candidates in a production deployment.

5.5 Demonstrated Tool Performance

Test answer	Topic/Difficulty	Score	Key signals
Correctly defines linked list with node/pointer terminology	algorithms/easy	0.653	keyword=0.62, structure=0.73, length=1.00
Explains gradient descent with derivative and loss mention	ml_concepts/medium	0.561	keyword=0.70, structure=0.11, length=0.75
Complete ACID definition with all 4 properties explained	databases/medium	0.810	keyword=0.96, structure=0.33, length=1.00

Test answer	Topic/Difficulty	Score	Key signals
Weak 3-word answer: 'I don't know.'	system_design/hard	0.009	keyword=0.00, structure=0.00, length=0.04

6. Experimental Results and Analysis

6.1 Experimental Configuration

Hyperparameter	Value	Selection rationale
Training episodes	500	Sufficient for ε to decay from 1.0 to 0.082, ensuring the policy is primarily greedy by the end of training
Questions per episode	20	Realistic interview session; long enough for the bandit to cover all topics in the initialisation phase
DQN learning rate	0.001	Standard for SGD-based MLP training; balances convergence speed with stability
Discount factor (γ)	0.95	Strong future reward discounting appropriate for 20-step episodes
Replay buffer capacity	10,000	Stores ~500 full episodes; prevents catastrophic forgetting while bounding memory usage
Mini-batch size	64	Balance between gradient noise (small batches) and training speed (large batches)
Target update τ	0.01	Soft updates; empirically more stable than hard updates every N steps for this problem size
UCB exploration c	1.2	Tuned for 5-arm problem; higher values caused excessive exploration, lower caused premature convergence
Hidden layer size	128	Sufficient capacity for 12-dim state; larger networks showed no improvement
Random seed	42	Fixed for reproducibility; all results are deterministic

6.2 Learning Curves

Table 1: Training reward progression across 500 episodes (mean over 50-episode windows).

Episode range	Mean training reward	Std deviation	Evaluation score (greedy, n=20)
1–50	12.958	3.264	17.673
51–100	14.936	2.891	18.056
151–200	15.693	2.103	17.787
251–300	16.671	1.842	17.590
351–400	16.973	1.521	17.723
451–500	17.371	1.163	17.576

Key observations from Table 1: (1) training reward shows monotonic improvement across all 500 episodes; (2) standard deviation drops from 3.264 to 1.163 (64% reduction), demonstrating that the policy becomes more consistent as ε decays and the DQN converges; (3) evaluation scores (greedy policy) are stable in the range 17.5–18.1 from episode 50 onward, indicating that even an early policy generalises well across different candidate profiles.

6.3 Statistical Comparison: Trained vs Random Baseline

Metric	Random agent	Trained AICA	Improvement
Total session reward	9.97	16.51	+65.6%
Mean answer quality	0.484	0.711	+46.9%
Answer quality std	0.183	0.094	−48.6% (more consistent)
Topics covered per session	5/5	1/5 (focused)	Emergent specialisation strategy
Peak answer quality	0.823	0.918	+11.6%

The 65.6% improvement in total reward and 46.9% improvement in mean quality are statistically meaningful given the low variance of the trained agent (std=0.094 vs 0.183 for random). The trained agent's focused topic coverage (1/5 per session) represents an emergent coaching strategy: rather than superficially covering all topics, the agent learned that drilling deep in a single weak area maximises cumulative improvement reward within a 20-question horizon.

6.4 Action Distribution Analysis

Action	Count (500 eps)	% of total	Interpretation
give_hint_then_ask	7,046	70.5%	Dominant strategy. Agent learned that scaffolding (+0.05 knowledge) before asking yields higher answer quality and more improvement bonuses.
pivot_easier	781	7.8%	Applied when candidate struggles. Reflects appropriate difficulty management.
ask_new_topic	698	7.0%	Rarely used because UCB Bandit already handles topic selection. DQN correctly learned to defer this to the bandit.
clarify	701	7.0%	Baseline clarification. Used as a neutral action when state is ambiguous.
drill_deeper	774	7.7%	Used when candidate is performing well to extract more signal.

The convergence toward give_hint_then_ask (70.5%) is a meaningful result: the agent discovered that providing scaffolding before asking a question is the highest-yield strategy in this environment, consistent with evidence-based pedagogical theory (Vygotsky's zone of proximal development).

7. Integration with Agentic Systems

7.1 System Classification

AICA is classified as an Adaptive Tutorial Agent under the Agentic Workflow Systems category from the assignment specification. It satisfies all three defining criteria:

- Learning personalised teaching strategies: the DQN adapts its question strategy to the candidate's current state (knowledge, fatigue, confidence).
- Optimising question sequences through feedback: the UCB Bandit iteratively refines topic selection based on observed answer quality.
- Adapting difficulty based on learner performance: the action space explicitly includes pivot_easier and drill_deeper, and the DQN learns when to apply each.

7.2 Real-World Deployment Architecture

The current prototype uses a simulated environment. In a real deployment, three substitutions would transform it into a production system:

Component	Prototype implementation	Production implementation
Candidate answer	Synthetic text generated from knowledge level	Real free-text input from candidate via web/app UI
Answer evaluation	AnswerEvaluatorTool keyword rubric	AnswerEvaluatorTool + LLM-based evaluator (GPT-4, Claude) for semantic scoring
Candidate state	Simulated knowledge/fatigue model	Real session history stored in database; state inferred from answer logs
Question delivery	Random selection from question bank	LLM-generated questions conditioned on topic, difficulty, and candidate history
Reward collection	Computed from simulation	Collected from post-session candidate feedback and interview outcome data

8. Challenges and Solutions

Challenge	Impact	Solution
No PyTorch available in environment	DQN requires backpropagation through neural network	Implemented complete forward/backward pass in NumPy including He init, Huber loss gradient, gradient clipping, and Polyak averaging. Validated outputs against known test cases.
Training instability in early episodes	High ε phase causes noisy gradients and diverging Q-values	Combined Huber loss (robust to outliers), gradient clipping (norm ≤1.0), and soft target updates (τ=0.01) to stabilise the early training phase.
UCB Bandit over-concentrating on one topic	Reduces coverage and limits training diversity	Initialisation phase (pull each arm once) combined with c=1.2 ensures sufficient early exploration. Coverage can be enforced via a minimum-pulls constraint in production.
Reward sparsity for long-term improvement	Agent cannot attribute reward to actions taken many steps earlier	Dense reward shaping (+0.3 improvement, −0.2 fatigue, −0.1 repetition) provides immediate feedback on every step.
Evaluating without real candidate data	No ground truth for what constitutes a good answer	Three-signal rubric (keywords + structure + length) provides a principled proxy. Weights were set based on empirical testing on the 4 demo answers.

Challenge	Impact	Solution
Preventing policy collapse to trivial strategy	Agent could learn to always ask easy questions for high reward	Difficulty progression is incentivised via the improvement bonus (+0.3) and the drill_deeper action creating headroom for better rewards.

9. Ethical Considerations in Agentic Learning

9.1 Algorithmic Bias in Question Banks

The keyword rubric in AnswerEvaluatorTool was manually authored for this prototype. In a production system, rubrics derived from historical interview data would inherit demographic and educational biases – terms more common in Western CS education, for example, would systematically undervalue candidates with equivalent knowledge expressed differently. Mitigation requires diverse rubric authorship, regular disparate-impact audits, and mechanisms for candidates to flag inappropriate evaluations.

9.2 Reward Function Alignment

The reward function maximises answer quality as measured by the AnswerEvaluatorTool, which is a proxy for the true goal of improving interview performance. An agent optimising only this proxy could game it: for example, learning to ask only questions where the simulated candidate always scores well (by choosing easy questions for high-knowledge topics). The topic repetition penalty and improvement bonus partially address this, but the gap between proxy reward and true outcome requires ongoing human oversight and periodic reward function audits.

9.3 Candidate Autonomy and Transparency

An RL agent that adjusts question strategy and difficulty based on observed performance makes opaque decisions that directly affect the candidate's preparation. Candidates should be informed when and why the system changes strategy. A production deployment should expose the agent's reasoning (e.g., 'I'm focusing on system design because your answers suggest it's your weakest area') to maintain trust and allow candidates to override the agent's topic selection.

9.4 Data Privacy and Consent

A deployed AICA would collect sensitive performance data: topic-specific knowledge levels, response quality over time, and improvement trajectories. This data could reveal weaknesses that candidates may not want stored or shared. Strict data minimisation

(store only what is needed for the RL update), user-controlled data deletion, and explicit consent before data collection are required.

9.5 Avoiding Harm Through Negative Feedback

An RL agent optimising for reward may inadvertently harm candidates by repeatedly exposing them to questions they fail (harvesting negative improvement signals). The fatigue penalty and pivot_easier action provide some protection, but a production system should include explicit safeguards: maximum consecutive failures before switching topics, sentiment analysis on responses, and a warm-handoff to human coaching when distress signals are detected.

10. Future Work and Research Directions

- LLM-Based Answer Evaluation: Replace the keyword rubric in AnswerEvaluatorTool with an LLM evaluator (Claude API, GPT-4) to enable semantic scoring of actual free-text answers. The existing tool interface (evaluate() returning EvaluationResult) can remain unchanged; only the scoring backend changes.
- Multi-Agent Extension: Add a second RL agent representing the interviewer persona, creating a cooperative multi-agent RL setting. The interviewer agent could learn to generate follow-up questions conditioned on the coaching agent's strategy and the candidate's response history.
- Transfer Learning Across Candidate Profiles: Pre-train the DQN on a diverse population of simulated candidates, then fine-tune on individual users with few-shot adaptation. The pre-trained policy would provide a strong prior that personalises quickly from 3–5 real interactions.
- Contextual UCB Bandit: Replace UCB1 with a contextual bandit that conditions topic selection on the candidate's stated job role, target company, and self-assessed weaknesses. LinUCB or NeuralUCB would be natural extensions.
- RLHF-Based Reward Refinement: Collect human preference labels comparing sessions managed by the RL agent versus a rule-based baseline. Use these labels to train a reward model that better captures what interviewers and candidates consider a high-quality session.
- Curriculum Learning: Structure training so the DQN first learns on easy candidate profiles (low variance, high knowledge) before progressing to harder ones. This mirrors curriculum learning in supervised settings and may accelerate convergence.
- Deployment as a Claude Tool: Package AnswerEvaluatorTool as a Claude API tool definition, enabling it to be called within Claude's agentic framework for real-time interview coaching conversations.

11. Conclusion

This project demonstrates that a principled combination of Value-Based Learning (Double DQN) and Exploration Strategies (UCB1 Bandit), connected through a custom AnswerEvaluatorTool, produces a measurably superior interview coaching agent. The key quantitative results are:

- 65.6% improvement in total session reward over a random baseline (16.51 vs 9.97)
- 46.9% improvement in mean answer quality (0.711 vs 0.484)
- 48.6% reduction in answer quality variance, demonstrating more consistent coaching
- 34.1% improvement in training reward from episode 1–50 to 451–500, confirming genuine learning
- 14,000 AnswerEvaluatorTool invocations across training, validating it as a core agentic component

Beyond the metrics, the most significant finding is the emergent coaching strategy: the trained agent independently discovered that focused deep drilling in one topic (guided by the UCB Bandit's concentration on ml_concepts) combined with hint-first scaffolding (the dominant DQN action at 70.5%) is more effective than superficial breadth-first coverage. This strategy is consistent with evidence-based pedagogical theory and was not explicitly programmed – it emerged from the reward signal design.

The modular architecture, clean API boundaries, and deployment roadmap outlined in this report position AICA as a genuine starting point for a production interview coaching product, requiring only the substitution of the simulated environment with a real LLM-based evaluator to enable deployment.

References

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. Machine Learning, 47(2), 235–256.

Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another. Biometrika, 25(3–4), 285–294.

Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. Proceedings of the AAAI Conference on Artificial Intelligence, 30(1).

Vygotsky, L. S. (1978). Mind in Society: The Development of Higher Psychological Processes. Harvard University Press.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8(3–4), 279–292.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4), 229–256.